

Roadmap to 1.0 Beta

The agreed plan of record for closing the gap between what Stratafetch documents and what it implements. Items are ordered; earlier work de-risks later work.

Status legend: **todo**, **in progress**, **done**.

0. Unblock the Windows dev loop **[DONE]**

`npm run check` cannot pass on a Windows working tree. Global `core.autocrlf=true` gives every file CRLF on disk, the repo has no `.gitattributes` to override it, and Prettier defaults to `endOfLine: "lf"` - so `format:check` fails all 74 files while CI passes on Linux. `.editorconfig` already declares `end_of_line = lf`, so the intent is unambiguous.

Fix: add `.gitattributes` with `* text=auto eol=lf`, then `git add --renormalize ..` This touches every tracked file once and should land as its own isolated commit.

A second, independent Windows blocker: `package-lock.json` contains only the Linux platform packages for `@napi-rs/canvas` (`canvas-linux-x64-gnu`, `canvas-linux-x64-musl`) and no `canvas-win32-x64-msvc`. `npm ci` honours the lockfile strictly, so the native binding can never install on Windows. `pdf-parse` pulls `pdfjs-dist`, which needs that binding to polyfill `DOMMatrix`, so `apps/api` tests fail at import for every Windows contributor while CI stays green on Linux. The lockfile was evidently generated on Linux only. Regenerate it so all target platforms are represented, and consider a CI matrix entry on `windows-latest` to keep it that way.

Also observed: `node_modules` on the primary dev machine was production-pruned (devDependencies absent). `npm ci` restores it; the prune is not reproducible from the repo and needs no code change.

Resolved 2026-08-29. `.gitattributes` added (`* text=auto eol=lf`) and the working tree renormalized to LF, dropping Prettier failures from 74 files to 0. The two files that still differed were Prettier 3.9 union-type reflow, not line endings, and were formatted. The lockfile was repaired surgically: the 8 missing `@napi-rs/canvas` platform packages were spliced in from a clean resolve with zero changes to any existing pinned version (a full regenerate would have churned 47 versions including a `cookie` major, so it was rejected). `npm ci` now validates the lockfile, installs `canvas-win32-x64-msvc`, and `npm run check` exits 0 on Windows with all 27 tests passing. A **cross-platform** CI job (`windows-latest`, `macos-latest`) was added to guard against regression.

1. Align the scope model with the documented flow **[DONE]**

A `shape`-scoped key can queue work it can never read. `POST /v1/shapes` returns **202** and there is no `GET /v1/shapes/{id}`; the only way to reach the result is `GET /v1/operations/{id}`, which requires the `admin` scope. The README quickstart has the same defect: it issues a key scoped `fetch, survey, collect`, then instructs the reader to poll `/v1/operations/{id}`, which returns **403 INSUFFICIENT_SCOPE**.

- Let a capability scope read and cancel the operations it created, rather than requiring `admin` for every read.

- Add **GET /v1/shapes/{id}** for parity with Survey and Collection.
- Update **README.md** and **docs/api.md** to match whatever rule is chosen.

Resolved 2026-08-29. Authorization for **GET /v1/operations/{id}**, its **/export**, and **/cancel** is now per-operation-type: a key holding the scope that matches the operation's type (with **collection** → **collect**) may read, export, and cancel it, so a capability key can follow its own async work. Cross-cutting actions — the **GET /v1/operations** list, delete, key management, and **/metrics** — stay **admin**. A new **GET /v1/shapes/{id}** gives Shape the same typed read endpoint Survey and Collection already had. **README**, **docs/api.md**, and the generated OpenAPI were updated to match; five authorization tests were added.

2. Normalize the idempotent-replay envelope **[DONE]**

POST /v1/fetch and **POST /v1/search** return two different body shapes for the same endpoint. A fresh request returns **{operationId, data: {source, content, retrieval}}**; an **Idempotency-Key** replay returns **{data: <OperationRecord>}**. A client that retries a request it already made gets a structurally different response.

Replay should return the stored result in the same envelope as the original.

Resolved 2026-08-29. A shared **replayEnvelope** helper (**apps/api/src/operations/ replay.ts**) rebuilds each endpoint's success envelope from the stored operation: a fetch replay returns **{operationId, data}** and a search replay returns **{data: {operationId, results}}**, matching the fresh responses. A replay of a failed original re-raises its stored error (**502**), and a replay that races an unfinished original returns **409 IDEMPOTENCY_IN_PROGRESS**. Covered by three tests in **app.test.ts**; **docs/api.md** updated.

3. Harden and prove the egress path **[TODO]**

Not a correctness blocker. Verified on 2026-08-29: **NODE_USE_ENV_PROXY=1** genuinely routes global **fetch** through the proxy on Node v22.23.0 and v24.17.0, so squid plus the **internal: true** network is enforcing real egress control today. What remains is fragility, not breakage:

- Node emits **[UNDICI-EHPA] Warning: EnvHttpProxyAgent is experimental**. The egress security boundary depends on an experimental undici feature.
- **Dockerfile** uses the floating **node:22-bookworm-slim** tag and **engines** allows **>=22**. The env-proxy support was backported partway through the 22.x line, so an older 22.x would silently stop proxying. Nothing pins this.
- **fetch/service.ts** passes **proxy**: explicitly to the browser retriever while every other outbound call relies on the environment variable — two mechanisms for one boundary.

Actions:

- Set an explicit dispatcher (**ProxyAgent / EnvHttpProxyAgent** via **setGlobalDispatcher**) once at process start in both **server.ts** and **worker.ts**, so both retrieval paths share one mechanism and neither depends on env-var handling.

- Add **.node-version** and tighten **engines.node** to a floor that actually has the behaviour the deployment relies on.
 - Extend the **compose-smoke** CI job past **/health/ready** to issue a real **POST /v1/fetch** and assert content came back. No current test would catch a broken egress path.
-

4. Rewrite the robots.txt parser [TODO]

robots/service.ts has no tests, and robots compliance is a headline claim.

- Consecutive **User-agent:** lines forming one group drop rules. **User-agent: *** followed by **User-agent: Googlebot** then **Disallow: /x** sets **applies = false** and discards a rule that does apply to *****.
- **permitted()** uses **path.startsWith(rule.path)**, so ***** wildcards and **\$** anchors are matched literally. **Disallow: /*.pdf\$** matches nothing.
- **Crawl-delay** and **Sitemap:** directives are ignored entirely.

Rewrite against a test table covering group merging, wildcards, anchors, longest-match precedence, and the two ignored directives.

5. Backfill tests [TODO]

Seven test files for a security-sensitive crawler. Untested: the auth service, **OperationRepository** (the idempotency replay/conflict SQL is subtle), robots parsing, the survey processor, the shape processor, content extraction, the redirect chain in **http-retriever**, and the migration runner. **App.tsx** is 1,699 lines behind one smoke test. Prefer a Postgres service container so migrations are exercised too.

6. Reconcile docs/security.md with the implementation [TODO]

The doc instructs operators to "pin the validated public address for the connection to reduce DNS-rebinding exposure." **assertSafeHttpUrl** resolves DNS, checks the addresses, then discards them; **fetch** re-resolves independently. Either implement address pinning via a custom dispatcher or **lookup**, or amend the doc. As written it overstates the posture.

Backlog

Smaller items, no fixed order:

- OpenAPI omits the entire admin surface — **/v1/admin/session**, **/v1/admin/keys**, **/v1/admin/keys/{id}**, **/v1/admin/providers** — and **/metrics**. **openapi:check** passes because it compares against the same incomplete generator.
- Survey discovery reads only **/sitemap.xml**: no sitemap-index recursion, no **.gz**, no **Sitemap:** from robots.txt. That fetch is also the only outbound call in the codebase not preceded by **assertSafeHttpUrl**.

- BullMQ **attempts: 3** combined with processors that call **operations.fail()** and then rethrow means a failing Collection re-fetches every URL up to three times and the operation status flaps **failed -> running -> completed**.
- **Ajv.compile()** runs on caller-supplied JSON Schema, making user-supplied **pattern** regexes a ReDoS vector in the worker.
- **apps/web** pins TypeScript **^5.7.2** while the root pins **7.0.2**.
- The crawler identifies as **Stratafetch/0.1** in both retrievers but **Stratafetch/1.0** in the robots service, against a package version of **1.0.0-alpha.1**.
- The version string is hardcoded in **app.ts** and **openapi.ts** rather than read from **package.json**.